

I still remember my first time using ChatGPT during my junior year of university. I had asked the model to “find me the top 10 highest-rated restaurants in Virginia.” Within seconds, I was handed a comprehensive list complete with addresses, cuisines, and ratings. My immediate thought was, *Holy shit*.

That was just three years ago. Since then, the landscape has exploded as more companies have released their flagship large language models: Google’s Gemini, Anthropic’s Claude, xAI’s Grok, and OpenAI’s ChatGPT.

As artificial intelligence becomes more deeply integrated into our daily lives, the need to truly harness its power grows. But here’s the catch: an AI’s efficiency is only as competent as its driver. We’ve come a long way from simply asking for restaurant recommendations.

AI Requires Intention

Recently, we received access to Claude at work. I initially used it to migrate and generate security detections, and the model was astoundingly efficient, cutting my workflow by roughly 40%. While 40% might not sound crazy to some, it was my first time using a coding model in a professional setting. Shaving 24 minutes off an hour-long task left me amazed.

Naturally curious, I went home, got myself Gemini Pro, and started experimenting. My first project was a personal financial tracker. Originally a python application, I refactor it into a full web application built with FastAPI and SQLite. At first, everything worked perfectly. But as I layered on more features, the frontend began to unravel: buttons disappeared, graphs displayed skewed data, and at one point, a page reload resulted in a completely blank screen.

Frustrated, I vented to my manager the next day. His response completely shifted my perspective: *“You need to understand that AI can only do what you explicitly tell it to do. It doesn’t know what you want; it only knows what you prompt.”*

Take a simple feature, like a box displaying total expenditures across all my credit cards. To a human developer, the end goal is intuitive because we can easily picture what we want to. To an AI, it’s a minefield of ambiguities. Do you want to include *all* cards or just active ones? Should the total be a positive or negative number? What color should the text be? How often should the API key refresh to pull bank data? Where is this data being stored?

When you fail to provide strict boundaries, the AI starts making its own assumptions to fill in the gaps. I learned that slow and steady really does win the race. I began breaking tasks down into microscopic steps: *“Create the container for the total expenditure number, then wait for my next prompt.” “Place the container in the top left of the Spendings tab, then wait.” “Sum up the charges for the cards ending in XXXX and YYYY, but do not include pending charges. Repeat these instructions back to me.”* Who knew that failing to state the trivial was exactly what held the AI back from its full potential?

Building the Context Window

Armed with this new mindset of giving exact, step-by-step instructions, I jumped right back to the CLI at work, ready to build out SQL detections for various logs. I figured I was about to double my output.

I was wrong. It turns out, prompting "Create CrowdStrike detections" is nowhere near enough. I actually spent more time troubleshooting why the AI was generating ineffective alerts than I would have spent just writing them myself.

I quickly realized that an LLM is essentially a brilliant new hire. You can tell a new hire to write a detection, but if they have no lay of the land, they will fail. What specific endpoints are we targeting? What is the exact schema of our CrowdStrike logs? How frequently is this detection running?

Before starting any project, you have to build the context window. Whether you are using steering files to guide the model or subagents to delegate tasks, the AI needs that baseline knowledge. Now, if I want an agent to write detections, I start by setting the stage: *"You are a senior security analyst. Your goal is to write high-fidelity detections that minimize noise in our SOAR platform. Reference the 'crowdstrike.csv' schema for available columns, and stage your changes in this specific Git branch."*

Taking 15 or 20 minutes to explicitly define the project's context might feel slow, but it saves hours of frustrating troubleshooting later. Preventing the AI from guessing is a distinct skill, and it is one I'm still trying to hone.

The Road Ahead

Learning to leverage AI requires a completely new set of skills, running parallel to the already rapidly evolving world of cybersecurity. It helps to remember that AI is an amplifier, not an autopilot. It is not a silver bullet that can independently solve complex engineering problems from scratch; rather, it is a highly capable assistant that relies entirely on your direction, logic, and oversight. Knowing how to use AI is one thing. Knowing how to *drive* it effectively is the true competitive advantage.